CSP-203: Software Tools & Technologies Lab

Chapter – 2

Bash Scripting: Basics (Lecture – 1)

Dr. Anand M. Baswade
Indian Institute of Technology, Bhilai
anand@iitbhilai.ac.in

Contents

- ❑ Introduction
- ❑ History
- ❑ How to write bash command
- ❑ How to print a statement
- ❑ Variables
- ❑ Operators
- ❑ Test Expression Evaluation
- ❑ Operations on strings

What is bash scripting?

- The process of writing scripts (a series of commands) using the Bash shell (short for Bourne Again SHell), which is a command-line interpreter.

What is bash?

- Bash stands for Bourne Again SHell.
- It is a Unix shell and command language interpreter.
- Developed as a free software replacement for the original Bourne shell

Why bash Scripting?

- ✓ Automation
- ✓ Simplicity and accessibility
- ✓ Integration with command-line tools
- ✓ Efficient for system administration
- ✓ Ideal with light weight task
- ✓ Portability
- ✓ Easy prototyping



Bash scripting

History of Bash:

- 1989 **Created by**: **Brian Fox** for the **GNU Project**

- Currently it is being used in:
  - Linux and Unix Systems
  - macOS
  - WSL (Windows Subsystem for linux)
  - Cloud platforms (AWS, GCP, Azure)



❑ How to write bash command:

1. Create a Script file.

   Command: gedit iit.sh

2. Type:

   #! /bin/bash

   echo “Hello, World!”

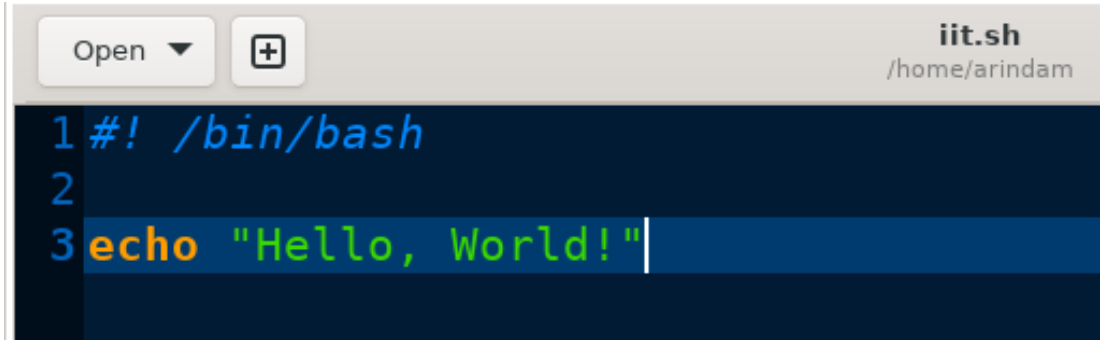
3. Save the file.

4. Make the file executable.

   Command: chmod +x iit.sh

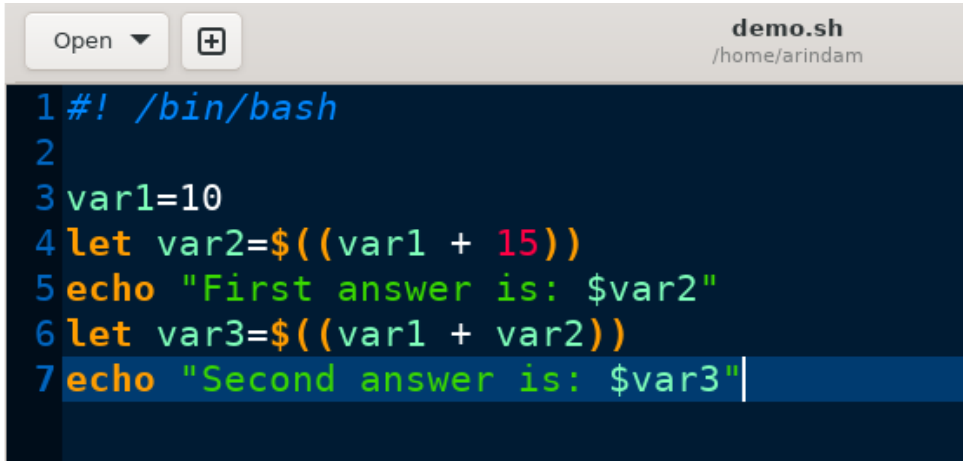
5. Execute/run the file

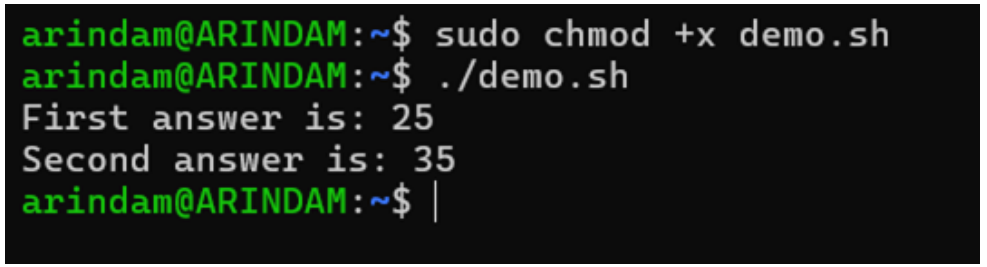
   Command: ./iit.sh




```
arindam@ARINDAM:~$ sudo chmod +x iit.sh
arindam@ARINDAM:~$ ls
iit.sh
arindam@ARINDAM:~$ ls -l
total 4
-rwxr-xr-x 1 root root 35 May 13 17:43 iit.sh
arindam@ARINDAM:~$ ./iit.sh
Hello, World!
arindam@ARINDAM:~$
```

- **echo command:**
  - It is used to print something on the terminal.
  - Works similar to the printf statement in C programming language
- **Variables: (Local Variables)**
  - To get anything done we need variables.
  - To read values in variables, precede their names by a dollar sign($).
  - We can print contents of any variable using the echo command.
- **let:**
  - It evaluates all expression following the equal sign.





- **Environment Variables:**
  - Used by system to define aspects of operations.
  - Examples:
    - ✓ $SHELL – which shell will be used by default.
    - ✓ $PATH – a list of directories to search for binaries.
    - ✓ $HOSTNAME – the hostname of the machine.
    - ✓ $HOME – current user's home directory.
- **How to a get all environment variables on terminal:**
  - Env command.

```
arindam@ARINDAM:~$ env
SHELL=/bin/bash
WSL2_GUI_APPS_ENABLED=1
WSL_DISTRO_NAME=Ubuntu-22.04
NAME=ARINDAM
PWD=/home/arindam
LOGNAME=arindam
MOTD_SHOWN=update-motd
HOME=/home/arindam
LANG=C.UTF-8
WSL_INTEROP=/run/WSL/264_interop
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=4
:ca=30;41:tw=30;42:ow=34;42:st=37;44:ex=01;
1;31:*.lzh=01;31:*.lzma=01;31:*.tlz=01;31:*
rz=01;31:*.lz=01;31:*.lzo=01;31:*.xz=01;31:
1:*.deb=01;31:*.rpm=01;31:*.jar=01;31:*.war
pio=01;31:*.7z=01;31:*.rz=01;31:*.cab=01;31
1;35:*.mjpeg=01;35:*.gif=01;35:*.bmp=01;35:
5:*.tiff=01;35:*.png=01;35:*.svg=01;35:*.sv
*.mkv=01;35:*.webm=01;35:*.webp=01;35:*.ogm
mv=01;35:*.asf=01;35:*.rm=01;35:*.rmvb=01;3
5:*.xwd=01;35:*.yuv=01;35:*.cgm=01;35:*.emf
id=00;36:*.midi=00;36:*.mka=00;36:*.mp3=00;
0;36:*.xspf=00;36:
WAYLAND_DISPLAY=wayland-0
LESSCLOSE=/usr/bin/lesspipe %s %s
TERM=xterm-256color
LESSOPEN=| /usr/bin/lesspipe %s
USER=arindam
DISPLAY=:0
SHLVL=1
XDG_RUNTIME_DIR=/run/user/1000/
WSLENV=
XDG_DATA_DIRS=/usr/local/share:/usr/share:/
PATH=/usr/local/sbin:/usr/local/bin:/usr/sb
 Files/NVIDIA GPU Computing Toolkit/CUDA/v1
c/Windows/system32:/mnt/c/Windows:/mnt/c/Wi
```

- **3 different types of quotes to enclose strings:**
  - **Single quotes(‘’):** Preserves the literal value of each character. A single quote may not occur between single quotes, even when preceded by a backslash.
  - **Double quotes(“”):** Preserves the literal value of all characters within the quotes, with the exception of $.
  - **Back quotes(``):** Executes the command with in the quotes.

```
#! /bin/bash

echo "$USER has given me $ 1.00"

echo '$USER has given me $ 1.00'

echo "I am $USER and today is `date`"
~
```

```
arindam@ARINDAM:~$ chmod +x demo.sh
arindam@ARINDAM:~$ ./demo.sh
arindam has given me $ 1.00
$USER has given me $ 1.00
I am arindam and today is Tue May 13 18:21:31 IST 2025
arindam@ARINDAM:~$
```

❑ Command Line Variables (Arguments):

- ➢ $1, $2, …, $10, $11 : are the values of the first, second, … arguments.
- ➢ $0: The name of the script.
- ➢ $# : The number of arguments
- ➢ $@: All the arguments, “$@” expands to “$1” “$2” … “$n”.

```
#! /bin/bash


echo "$1"
echo "$2"
echo "$0"
echo "$#"
echo "$@"
```

```
arindam@ARINDAM:~$ ./demo.sh 12 11
12
11
./demo.sh
2
12 11
arindam@ARINDAM:~$
```

❑ **Example of summation of 3 numbers through command line argument:**

```
#! /bin/bash

let x=$1+$2+$3
echo "The sum of three numbers is: $x"
~
```

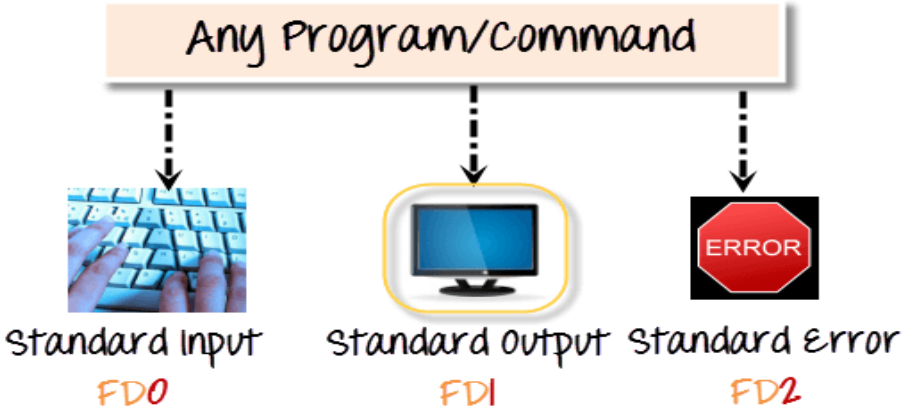
```
arindam@ARINDAM:~$ ./demo.sh 10 12 8
The sum of three numbers is: 30
arindam@ARINDAM:~$
```

Bash: Special Variables

Variable	Variable Description
$0	The filename of the current script.
$n	These variables correspond to the arguments with which a script was invoked where n is from 1,2….
$#	The number of arguments supplied to a script.
$*	All the arguments are double quoted. If a script receives two arguments, $* is equivalent to $1 $2
$@	All the arguments are individually double quoted. If a script receives two arguments, $@ is equivalent to $1 $2.
$?	The exit status of the last command executed
$$	The process number of the current shell. For shell scripts, this is the process ID under which they are executing.
$!	The process number of the last background command.

I/O Redirection

- *Redirection can be defined as changing the way from where commands read input to where commands sends output.*



File Descriptors on Linux

Name	Represents	Examples
File Descriptor 0 (fd[0])	Standard input	Keyboard, file, terminal
File Descriptor 1 (fd[1])	Standard output	Screen, database
File Descriptor 2 (fd[2])	Standard error	File, terminal

Symbol	Description
>	Directs the standard output of a command to a file. If the file exist, it is overwritten.
>>	Directs the output to a file, adding the output to the end of the existing file.
2>	Directs standard error to the file.
2>>	Directs standard error to a file, adding the output to the end of the existing file.
&>	Directs standard output and standard error to the file.
<	Directs the contents of a file to the command.
<<	Accepts text on the following lines as standard input.
<>	The specified file is used for both standard input and standard output.

- **Taking inputs manually:**
  - With the help of read command you can take input from the terminal.

```
#! /bin/bash

echo -n "Enter the number: "
read n

echo "Your entered number is: $n"
~
```

```
arindam@ARINDAM:~$ ./demo.sh
Enter the number: 20
Your entered number is: 20
```

Read

Used to take the user's input from the terminal

```
read: read [-ers] [-a array] [-d delim] [-i text] [-n nchars] [-N nchars] [-p prompt] [-t timeout] [-u fd] [name ...]
    Read a line from the standard input and split it into fields.

    Reads a single line from the standard input, or from file descriptor FD
    if the -u option is supplied.  The line is split into fields as with word
    splitting, and the first word is assigned to the first NAME, the second
    word to the second NAME, and so on, with any leftover words assigned to
    the last NAME.  Only the characters found in $IFS are recognized as word
    delimiters.

    If no NAMEs are supplied, the line read is stored in the REPLY variable.

    Options:
      -a array  assign the words read to sequential indices of the array
                variable ARRAY, starting at zero
      -d delim  continue until the first character of DELIM is read, rather
                than newline
      -e        use Readline to obtain the line in an interactive shell
      -i text   use TEXT as the initial text for Readline
      -n nchars return after reading NCHARS characters rather than waiting
                for a newline, but honor a delimiter if fewer than
                NCHARS characters are read before the delimiter
      -N nchars return only after reading exactly NCHARS characters, unless
                EOF is encountered or read times out, ignoring any
                delimiter
      -p prompt output the string PROMPT without a trailing newline before
                attempting to read
      -r        do not allow backslashes to escape any characters
      -s        do not echo input coming from a terminal
```

Usage: read **[options] [var1, var2, var3…]**
Common Options:

-p : Outputs the prompt string before reading user input.

```
#!/bin/bash
#Take the input with the prompt message
read -p "Enter the book name: " book
#Print the input value
echo "Book name: $book"
```

- **Operators:**
  - **Unary Operators:**
    - a++, a-- : Post-increment/decrement
    - ++a, --a : Pre-increment/decrement
  - **Arithmetic Operators:**
    - a+b, a-b : Addition/subtraction
    - a*b, a/b : Multiplication/division
    - a%b : Modulo
    - a**b : Exponential
  - **Relational Operators**:
    - a>b, a<b : Greater than, less than
    - a==b, a!=b : Equality, inequality
  - = : Assignments

```
#! /bin/bash

a=$1
echo "a after pre-increment: $((++a))"
~
```

```
arindam@ARINDAM:~$ ./demo.sh 30
a after pre-increment: 31
arindam@ARINDAM:~$
```

- **Operators:**
  - **Logical Operators:**
    - && : Logical AND
    - || : Logical OR
  - **Bitwise Operators:**
    - & : Bitwise AND
    - | : Bitwise OR
    - ^ : Bitwise XOR
    - << : Left shift
    - >> : Right shift

```
#! /bin/bash

echo "4|2 : $((4|2))"
echo "4&2 : $((4&2))"
echo "7^3 : $((7^3))"
echo "2<<4 : $((2<<4))"
echo "8>>2 : $((8>>2))"
```

```
arindam@ARINDAM:~$ ./demo.sh
4|2 : 6
4&2 : 0
7^3 : 4
2<<4 : 32
8>>2 : 2
```

Decision Making: If

```
if [<CONDITION1>]
      then
       <STATEMENT>
fi
```

if_condition.sh

```
#!/bin/bash

a=50
b=10

if [ $a -gt $b ]
  then
       echo "a is greater than b"
fi
```

-eq: equal
-ne: not equal
-gt: greater than
-lt: less than
-ge: greater than or equal
-le: less than or equal

Output:

```
$ chmod +x if_condition.sh
$ ./if_condition.sh

a is greater than b
```

Decision Making : If .. else

```
if [ <CONDITION> ]
        then
            <STATEMENT1>
        else
            <STATEMENT2>
         fi
```

if_elsecondition.sh

```
#! /bin/bash
a=110
b=48
if [ $a -lt $b ]
then
      echo "a is less than b"
else
      echo "a is greater than b"
fi
```

Output:

```
$ chmod  +x if_elsecondition.sh
$ ./if_elsecondition.sh

a is greater than b
```

Logical AND

Usage: command1 && command2

logical_and.sh

```
#!/bin/bash
n=38
if [ $((n % 2)) == 0 ] && [ $((n % 5)) == 0 ]
then
   echo "Number is  divisible by  both 2 and 5"
else
   echo "Number is not divisible by both of them"
fi
```

Output:

```
$ chmod +x logical_and.sh
$ ./logical_and.sh

Number is not divisible by both of them
```

Logical OR

Usage: command1 || command2

logical_or.sh

```
#!/bin/bash
num=38
if [[ $num%2 == 0 ] || [ $num%5 == 0 ]]
then
  echo "Divisible by either 2 or 5"
else
  echo "Not divisible by either of them"
fi
```

Output:

```
$ chmod +x logical_or.sh
$ ./logical_or.sh

Divisible by either 2 or 5
```

Logical NOT

Usage: !command1

logical_not.sh

```
#!/bin/bash

a=76
b=78

if [[ ! ${a} == ${b} ]]
then
   echo "both are not the same"
else
   echo "both are  the same"
fi
```

Output:

```
$ chmod +x logical_not.sh
$ ./logical_not.sh

both are not the same
```

- **Conditional Statements:**

```
If [ expression ]
then
    Statement 1
elif [ expression ]
then
    Statement 2
else
    Statement 3
fi
```

- **Test Expression Evaluation:**
  - ✓ n1 -eq n2 : tests if n1 == n2
  - ✓ n1 -ne n2 : tests if n1 != n2
  - ✓ n1 -lt n2 : tests if n1 < n2
  - ✓ n1 -le n2 : tests if n1 <= n2
  - ✓ n1 -gt n2 : test if n1 > n2
  - ✓ n1 -ge n2 : test if n1 >= n2
  - ✓ if either n1 or n2 is not a number then test fails.

- **Example:**

```
a=4
b=5
if [ $a -gt $b ]
then
    echo “a is greater than b”
else
    echo “b is greater than or equal to a”
fi
```

What is the output of the program?

- **Example:**

```
a=4
b=5
if [ $a -gt $b ]
then
    echo “a is greater than b”
else
    echo “b is greater than or equal to a”
fi
```

What is the output of the program?
=> b is greater than or equal to a

- **Example:**

```
a=2
b=5
if [ $a > $b ]
then
    echo “a is greater than b”
else
    echo “b is greater than or equal to a”
fi
```

What is the output of the program?

- **Example:**

```
a=2
b=5
if [ $a > $b ]
then
    echo “a is greater than b”
else
    echo “b is greater than or equal to a”
fi
```

What is the output of the program?
=> a is greater than b
Why?

- **Example:**

```
a=2
b=5
if [ $a > $b ]
then
    echo “a is greater than b”
else
    echo “b is greater than or equal to a”
fi
```

=> Relational operators works between strings only. Here, it means create a file $b and evaluate [$a] and it is true because of being non-empty string.

- **String Comparison:**
  - s1 == s2 : s1 and s2 are identical.
  - s1 != s2 : s1 and s2 are different.
  - s1 : s1 is not a null string.

```
#! /bin/bash

s1=$1
s2=$2

if [ $s1 == $s2 ]
then
        echo "Strings are equal!"
else
        echo "Strings are not equal!"
fi
```

```
arindam@ARINDAM:~$ ./demo1.sh IIT Bhilai
Strings are not equal!
arindam@ARINDAM:~$ ./demo1.sh IIT IIT
Strings are equal!
arindam@ARINDAM:~$
```

- **Example:**

```
#initializing two variables
a=“First”
b=“Second”
if [ “$a” == “$b”]
then
    echo “a is equal to b”
else
    echo “a is not equal to b”
fi
```

What is the output of the program?

- **Example:**

```
#initializing two variables
a="First"
b="Second"
if [ "$a" == "$b"]
then
    echo "a is equal to b"
else
    echo "a is not equal to b"
fi
```

What is the output of the program?

=> a is not equal to b

- **Example:**

```
#initializing two variables
a="First"
b="First"
if [ a == b]
then
    echo "a is equal to b"
else
    echo "a is not equal to b"
fi
```

What is the output of the program?

- **Example:**

```
#initializing two variables
a="First"
b="First"
if [ a == b]
then
    echo "a is equal to b"
else
    echo "a is not equal to b"
fi
```

Should be "a is equal to b"?

- **Example:**

```
#initializing two variables
a="First"
b="First"
if [ a == b]
then
    echo "a is equal to b"
else
    echo "a is not equal to b"
fi
```

Actual Output: "a is not equal to b"
Why?

- **Example:**

```
#initializing two variables
a="First"
b="First"
if [ a == b]
then
    echo "a is equal to b"
else
    echo "a is not equal to b"
fi
```

=> Variable names of a and b is being compared which are always different. Hence, condition is false, statement 2 will be printed.

Exercise:

Write a script that appends the contents of multiple files to a single file named “files.txt”. The names of the files should be provided as command-line arguments!

```
#! /bin/bash
output_file="files.txt"
if [ "$#" -ne 0 ]; then
        for fname in "$@"
        do
                if [ -f "$fname" ]; then
                        # to check if file is a regular file.
                        cat "$fname" >> "$output_file"
                else
                        echo "$fname does not exist or it is not a regular file"
                fi
        done
else
        echo "Please pass at least one file name"
fi
```

Exercise: Special variables

Write a script that accepts two integer numbers as command line arguments and displays their sum. If the number of arguments is not exactly 2, the script should output a message: "*Please provide exactly 2 arguments*." Additionally, if the provided arguments are invalid integer numbers, the script should display the message: "Please provide valid integer numbers."

```
#!/bin/bash

if [ $# -ne 2 ]; then
  echo “Please provide exactly 2 arguments”
exit 1
fi
x=$1
y=$2
sum=`expr  $x  +  $y`
if [ $? -ne 0 ]; then
  echo “Please provide valid integer numbers”
exit 2
else
echo “The sum: $sum"
fi
```

CASE STATEMENT

```
case EXPRESSION in
   Pattern_Case_1)
                STATEMENTS
            ;;
   Pattern_Case_1)
                STATEMENTS
            ;;
   Pattern_Case_N)
                STATEMENTS
            ;;
           *)
                STATEMENTS
            ;;
         esac
```

case.sh

```
#!/bin/bash
DEPARTMENT="Computer Science"
echo -n "Your DEPARTMENT is "
case $DEPARTMENT in

 "Computer Science")
        echo -n "Computer Science"
   ;;
 "Electrical and Electronics Engineering" | "Electrical
Engineering")
        echo -n "Electrical and Electronics Engineering or
Electrical Engineering"
   ;;

 "Information Technology" ")
        echo -n "Information Technology"
   ;;

  *)
        echo -n "Invalid"
   ;;
esac
```

Output:

```
$ chmod +x case.sh
$ ./case.sh
Your DEPARTMENT is Computer Science
```

While

```
while   [ CONDITION ]
          do
        <STATEMENT>
         done
```

while.sh

```
#!/bin/sh
a=0
while [ $a -lt 10 ]
do
    echo $a
    a=`expr $a + 1
done
```

Output:

```
$ chmod +x while.sh
$ ./while.sh
0
1
2
3
4
5
6
7
8
9
```

FOR

```
for <var> in <value1 value2 ... valuen
                    do
                    <command 1>
                     <command 2>
                   done
```

for.sh

```
#!/bin/bash
for i in 1 2 3 4 5
do
  echo "Welcome $i times"
done
```

Output:

```
$ chmod +x for.sh
$ ./for.sh
Welcome 1 times
Welcome 2 times
Welcome 3 times
Welcome 4 times
Welcome 5 times
```

FOR Loop with Break and Continue Statement

for_break.sh

```
#!/bin/bash
for a in 1 2 3 4 5 6 7 8 9 10
do
  if [ $a == 6 ]
   then
      break
  fi
    echo "Iteration no $a"
done
```

Output:

```
$ chmod +x for_break.sh
$ ./for_break.sh

Iteration no 1
Iteration no 2
Iteration no 3
Iteration no 4
Iteration no 5
```

for_continue.sh

```
#!/bin/bash
for a in {1..10}
do
   if [ $((a % 2)) -eq 0 ]
   then
      continue
   fi
   echo "Iteration no $a"
done
```

Output:

```
$ chmod +x for_continue.sh
$ ./for_continue.sh

Iteration no 1
Iteration no 3
Iteration no 5
Iteration no 7
Iteration no 9
```

Thank you!

Email: anand@iitbhilai.ac.in